Module 09 Lab - Ensemble Methods**Objective:** To learn how to use **ensemble methods**, which combine the predictions of several individual models to produce a final prediction that is more accurate and robust than any of the individual models alone.**In this lab, you will train and compare a single Decision Tree against a powerful ensemble method, the Random Forest.**

Part 1: The Wisdom of the Crowd**Concept:** The core idea behind ensemble methods is the "wisdom of the crowd." If you ask one person a difficult question, they might be wrong. But if you ask a large, diverse group of people the same question and average their answers, the result is often surprisingly accurate.Ensemble methods apply this to machine learning. Instead of relying on a single model, we build a "committee" of models and have them vote on the final prediction.Two popular ensemble techniques:* **Bagging (Bootstrap Aggregating):** Trains multiple models (e.g., Decision Trees) on different random subsets of the training data. The **Random Forest** is the most famous example.* **Boosting:** Trains models sequentially, where each new model focuses on correcting the errors made by the previous one. Examples include Gradient Boosting and AdaBoost.

Part 2: SetupWe will use the Iris dataset to classify flower species.

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load the Iris flower dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the data into training and testing groups
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
Training samples: 105
Testing samples: 45
```

Part 3: A Single Decision TreeA single Decision Tree is prone to overfitting. It can create very specific rules that match the training data perfectly but don't generalize well.**Your Task:** Train a single `DecisionTreeClassifier` and evaluate its accuracy.

```
# Create and train a single Decision Tree
single_tree = DecisionTreeClassifier(random_state=42)
single_tree.fit(X_train, y_train)
```

```
# Make predictions using the test data
y_pred_tree = single_tree.predict(X_test)

# Check the accuracy
accuracy_tree = accuracy_score(y_test, y_pred_tree)

print(f"Accuracy of a single Decision Tree: {accuracy_tree:.2%}")
```

```
Accuracy of a single Decision Tree: 100.00%
```

Part 4: The Random Forest Ensemble**Concept:** A Random Forest is an ensemble of many Decision Trees. It builds hundreds of trees, each trained on a random subset of the data and using a random subset of the features. To make a prediction, it gets a vote from every tree in the "forest" and chooses the class that gets the most votes.This process of averaging the votes of many diverse trees makes the Random Forest much more robust and less prone to overfitting than a single tree.**Your Task:** Train a `RandomForestClassifier` and compare its accuracy to the single tree.

```
# Create a Random Forest with 100 trees
random_forest = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train the Random Forest
random_forest.fit(X_train, y_train)

# Make predictions
y_pred_rf = random_forest.predict(X_test)

# Check the accuracy
accuracy_rf = accuracy_score(y_test, y_pred_rf)

print(f"Accuracy of the Random Forest: {accuracy_rf:.2%}")
```

```
Accuracy of the Random Forest: 100.00%
```

Part 5: Feature Importance**Concept:** A great benefit of tree-based ensembles like Random Forest is that they can tell you which features were most important for making accurate predictions.**Your Task:** Extract and plot the feature importances from your trained Random Forest model.

```
import matplotlib.pyplot as plt
import numpy as np

# Get the importance of each flower feature
importances = random_forest.feature_importances_

# Get the feature names
feature_names = iris.feature_names

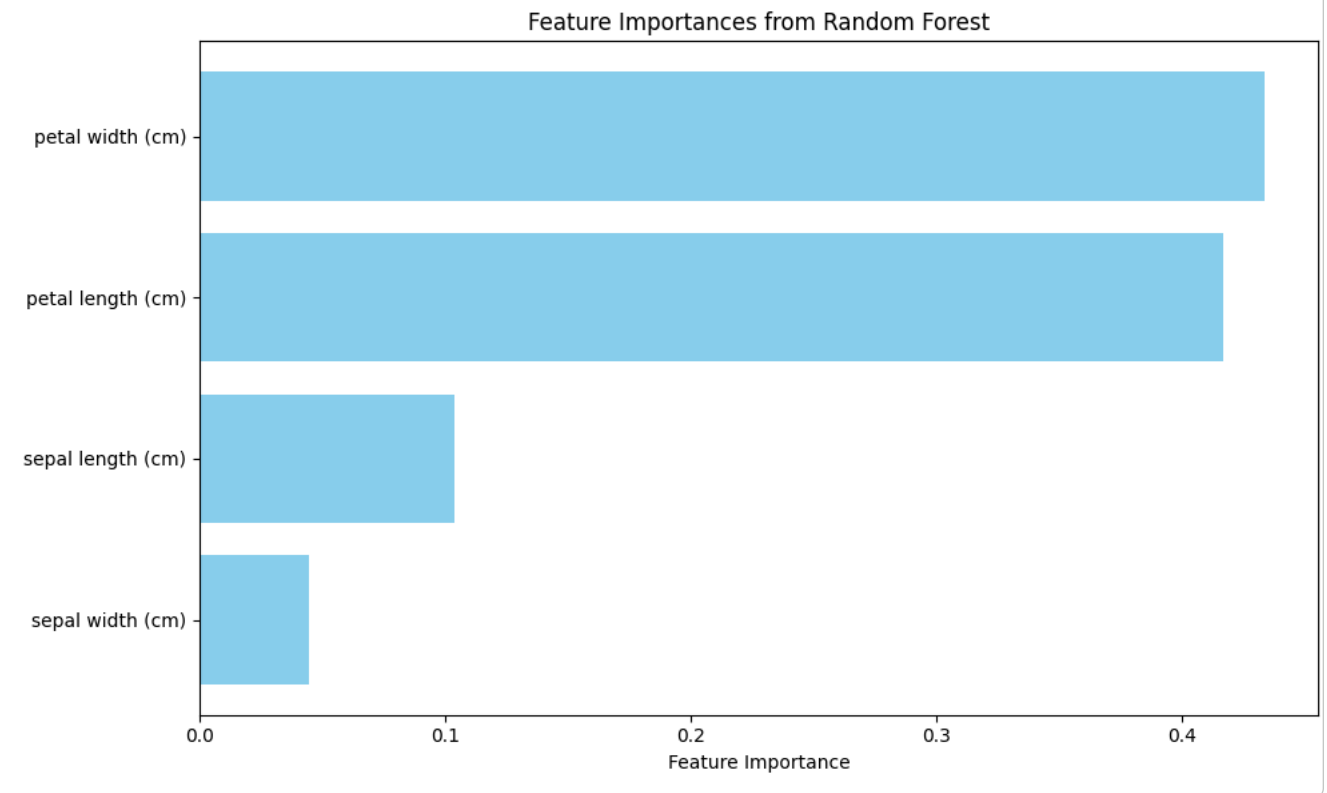
# Sort the features by importance
indices = np.argsort(importances)

# Create the graph
plt.figure(figsize=(10, 6))
plt.title("Feature Importances from Random Forest")

plt.barh(
    range(len(indices))
```

```
    importances[indices],
    color="skyblue"
)

plt.yticks(range(len(indices)), np.array(feature_names)[indices])
plt.xlabel("Feature Importance")
plt.tight_layout()
plt.show()
```



Knowledge Check**Instructions:** Answer the following questions in this markdown cell.

1. **In your own words, what is the main idea behind ensemble methods? Why is a "committee" of models often better than a single expert model?** Ensemble methods combine several models to make a final prediction. Instead of depending on just one model, they use multiple models to help reduce mistakes. I think this is useful because different models may notice different patterns, making the final prediction more reliable.
2. **Which model performed better in this lab, the single Decision Tree or the Random Forest? Is this what you expected? Why?** Both the Decision Tree and Random Forest achieved 100% accuracy in this lab. I expected Random Forest to perform better because it combines many trees, but both models gave the same result. This helped me understand that using more models does not always mean getting a higher accuracy, especially when the dataset is simple.
3. **Looking at the feature importance plot, which two features were most important for classifying the iris flowers?** Does this match what you might expect from your own knowledge of flowers?** The two most important features were petal width and petal length. This makes sense to me because flowers can have different petal sizes and shapes. I learned that these measurements helped the model identify the different Iris species more than the sepal measurements.